

Lecture 4 | k-Anonymity and an Introduction to Machine Learning

Wednesday, September 2, 2026

Instructors: Steven Wu, Giulia Fanti

In this lecture. This lecture introduces k -anonymity: a concrete privacy criterion for a published table, meant to block linkage attacks. We then break it twice, first with side information and then with a second k -anonymous release of the same population, and draw the general lesson that constraining the *form* of a release does not constrain the *information* it carries. The rest of the lecture opens the *Privacy and Machine Learning* unit: a model trained on personal data is itself a kind of aggregate release, and before we can ask what it leaks, we need a shared vocabulary for what a model is, how it is trained, and how the transformer architecture works.

1 Two Ways to Define “Privacy”

Lectures 2 and 3 showed that overly accurate answers to too many statistics reconstruct a dataset. That is a fact about *attacks*, not yet a *definition* of privacy, and there are two natural ways to get from one to the other.

1. **Think of possible attacks, and build defenses against them.** Enumerate the ways an adversary might misuse a release, then design a criterion that specifically blocks those attacks. k -anonymity, which we look at next, is exactly this kind of answer, aimed at *linkage attacks*.
2. **Formulate a general criterion** that does not target any one attack, and argue that it rules out an entire broad class of them at once.

The first approach is concrete and easy to check, but it is only ever as good as the list of attacks it was designed against. Watching k -anonymity fail against attacks it was *not* designed for is the cleanest motivation this course has for the second approach, which we return to once the machine-learning unit is behind us.

2 k -Anonymity

k -anonymity [5] applies to *generalized microdata*: a table of individual records in which each entry is either the original value or a set of possible values that contains it.

Definition 2.1. A *generalization* of a table replaces a single value with a set of possible values consistent with it — for example, collapsing Male, Female, Other into the single category $\{*\}$, or replacing an exact age with a range such as <30 , $30\text{--}39$, or ≥ 40 .

Attributes are split into *non-sensitive* ones, assumed already available to an attacker (zip code, age, nationality), and *sensitive* ones, assumed unknown (a medical condition, say).

Definition 2.2. A table is *k-anonymous* if every row, restricted to its non-sensitive attributes, matches at least $k - 1$ other rows.

Table 1 is 4-anonymous: on Zip code, Age, and Nationality, the twelve rows fall into three groups of four identical entries.

	Non-Sensitive			Sensitive
	Zip code	Age	Nationality	Condition
1	130**	<30	*	AIDS
2	130**	<30	*	Heart Disease
3	130**	<30	*	Viral Infection
4	130**	<30	*	Viral Infection
5	130**	≥40	*	Cancer
6	130**	≥40	*	Heart Disease
7	130**	≥40	*	Viral Infection
8	130**	≥40	*	Viral Infection
9	130**	3*	*	Cancer
10	130**	3*	*	Cancer
11	130**	3*	*	Cancer
12	130**	3*	*	Cancer

Table 1: A 4-anonymous table.

Exercise 2.3. *Think-pair-share.* Make the following table 2-anonymous. Unlike Table 1, treat every column as fair game to generalize — there is no designated sensitive attribute here.

Name	Age	Gender
Alice	12	F
Bob	22	M
Charlie	23	M
David	15	M

There is no unique answer — *k-anonymity* only asks that some generalization of every column make the rows collide in groups of k . One valid choice generalizes Age to <20 vs. ≥20, Gender to {F,M}, and Name to whether its length is 5 (Alice, David) or not (Bob, Charlie):

Name	Age	Gender
len = 5	<20	{F,M}
len = 5	<20	{F,M}
len ≠ 5	≥20	{F,M}
len ≠ 5	≥20	{F,M}

Two groups of two, matching on every generalized column: this is a valid 2-anonymous release of the table.

2.1 What Can Go Wrong

The stated goal is to resist *linkage attacks*: an attacker who cannot single out a record by its non-sensitive attributes should not be able to link it to an outside data source either, since at least k other records look identical on those attributes. Table 1 does seem to resist this — no combination of zip code, age, and nationality picks out fewer than four people.

But the same table still leaks. Everyone between 30 and 39 in Table 1 has cancer, whether or not you can tell which row is which. If you already know that Alice, who is 32 and lives in zip 130—, is in this table, you now know she has cancer, with no record ever singled out. k -anonymity says nothing about this, because it only constrains *how many* rows share a combination of non-sensitive attributes, not what the sensitive attribute looks like within that group.

k-anonymity is not robust to side information. Knowing that everyone in a group shares a sensitive value defeats the guarantee even when no individual row is ever identified.

2.2 Composition

A second, independent failure comes from *composition*: the same population anonymized twice, for two different releases, can be cross-referenced [1]. Table 2 is a second, 6-anonymous release of the same population as Table 1, generalized on a different age boundary.

		Non-Sensitive		Sensitive
	Zip code	Age	Nationality	Condition
1	130**	<35	*	AIDS
2	130**	<35	*	Tuberculosis
3	130**	<35	*	Flu
4	130**	<35	*	Tuberculosis
5	130**	<35	*	Cancer
6	130**	<35	*	Cancer
7	130**	≥35	*	Cancer
8	130**	≥35	*	Cancer
9	130**	≥35	*	Cancer
10	130**	≥35	*	Tuberculosis
11	130**	≥35	*	Viral Infection
12	130**	≥35	*	Viral Infection

Table 2: A 6-anonymous release of the *same* population, from a different data collection with a different age generalization.

Suppose an attacker knows that Alice is 28 years old, lives in zip 130—, and appears in both tables. In Table 1 alone, being under 30 narrows her condition to {AIDS, Heart Disease, Viral Infection}. In Table 2 alone, being under 35 narrows it to {AIDS, Tuberculosis, Flu, Cancer}. Neither release identifies her condition on its own. But she is in both groups at once, so her condition must lie in the *intersection*

of the two possibility sets — which is exactly {AIDS}. Two anonymous releases, each individually safe by the k -anonymity criterion, together pin her down exactly.

2.3 Form versus Process

Both failures share a root cause. k -anonymity *specifies a set of acceptable output tables* — any table where non-sensitive attributes repeat in groups of k passes — but it does not specify, or restrict, the *process* that produced that table. That flexibility is not neutral: which generalization the curator chooses can itself leak information (for instance, if the choice depends on the sensitive column), and nothing in the criterion prevents two independently “safe” releases of the same population from being combined.

Restricting the form of a release — how many times each combination of non-sensitive attributes occurs — does not restrict the information the release carries. A privacy definition that constrains the process producing a release, rather than only the shape of its output, is what the second half of this course builds toward.

Later refinements patch the side-information failure specifically — ℓ -diversity [3] requires every group to contain several distinct sensitive values, and t -closeness [2] requires the distribution of the sensitive attribute within each group to resemble the overall distribution. Both inherit the composition failure, since neither constrains what happens when a population is anonymized more than once. We will not develop either fix further; the pattern of patching syntactic definitions against one new attack at a time is the point, not the specific fix.

3 A New Unit: Privacy and Machine Learning

The rest of the course asks the same question — what does a release reveal about the people in the data — about a different kind of release: a trained machine learning model. A model is a function learned from a dataset, and, just as with the statistical releases of the last three lectures, we will ask whether that function reveals more about its training data than intended. Before we can ask that, we need to agree on what a model is and how it comes to be.

3.1 The Machine Learning Pipeline

A *data holder* collects or purchases a dataset $\mathbf{x}$ from individual users, and trains a model $\mathcal{M}$ on it, represented by parameters w . An *analyst* then queries the trained model with new inputs and receives its outputs — for instance, a chatbot prompted with “Hi, are you home?” answering with one of home, happy, tired.

The analyst never sees $\mathbf{x}$ directly, only $\mathcal{M}$ (or its outputs). That gap is exactly what a privacy attack tries to close. Two questions organize the next lecture’s attacks, and are worth stating now:

1. **Membership.** Was a particular user U ’s data part of the training dataset $\mathbf{x}$?
2. **Property inference.** Can an analyst infer properties of the data entries in $\mathbf{x}$ from $\mathcal{M}$ alone?

3.2 What Is a Machine Learning Model

Definition 3.1. A machine learning model is a function $\mathcal{M}$ mapping an input x to an output $\mathcal{M}(x)$. *Training* is the process of choosing which function $\mathcal{M}$ to use so that it fits the training data; *inference* is running the trained $\mathcal{M}$ on a new input x .

Models split into two broad families, distinguished by what they take in and what they hand back.

	Discriminative	Generative
Input	a sample (image, text, features)	noise, optionally a condition (e.g. text)
Output	a categorical label or numeric prediction	a sample (text, image, video)
Examples	classifier, regression	GAN, diffusion model, autoregressive transformer

Table 3: Discriminative models predict a label from a sample; generative models produce a sample from noise.

3.3 What Determines a Model's Privacy Behavior

Three choices, made before and during training, determine what a finished model can reveal about the data it was trained on:

- **The training data itself** — a model can only leak what was in x to begin with.
- **The model architecture**, which fixes the *function class* $\mathcal{M}$ is chosen from — a linear model, a convolutional network, a transformer. A larger, more expressive class can fit (and memorize) its training data more closely.
- **The loss function**, which specifies how the training data is used to pick one function out of that class.

The rest of this section makes these three choices concrete with a worked example on each side of Table 3.

3.4 A Discriminative Example: Linear Regression

Fitting tip amount from bill size

Data. n pairs (x_i, y_i) , $x_i \in \mathbb{R}^d$ input features and $y_i \in \mathbb{R}$ a numeric target — here, x_i is the size of a restaurant bill and y_i the tip left on it.

Architecture (linear). Restrict $\mathcal{M}$ to functions of the form

$$\mathcal{M}(x) = w^\top x, \quad w \in \mathbb{R}^d,$$

so training only has to choose the d numbers in w .

Loss function (mean squared error). Score a candidate w by how far its predictions land

from the true tips,

$$\ell(x, y) = (\mathcal{M}(x) - y)^2 = (w^\top x - y)^2.$$

Training. Choose the w that minimizes the loss averaged over the training data:

$$\hat{w} \in \arg \min_{w \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^n (y_i - w^\top x_i)^2.$$

This is the smallest possible instance of the three choices above: an architecture (linear functions of x), a loss (squared error), and a dataset, combined by an optimization problem into one fitted model $\hat{w}$.

3.5 How Generative Models Are Trained

Generative models are trained differently. There is no label y to predict — the training data is a collection of examples *drawn from the target distribution* itself (photographs, if the goal is to generate photographs), and training fits a function that turns noise (and, often, a condition such as a text prompt) into new samples from that same distribution. The available architectures and matching loss functions include transformers, diffusion models, GANs, and state-space models; we develop the details of specific architectures as we need them.

3.6 Transformer Self-Attention, a First Look

The transformer [6] is the architecture behind essentially every large discriminative and generative language model in current use, so it is worth a first look now, even though we will not need its full mechanics until we get to attacks that exploit it.

Its original use case was translation. Consider

English: Can you help me translate this sentence?

German: Kannst du mir helfen diesen Satz zu uebersetzen?

The two sentences do not align word by word — German *helfen* (“help”) corresponds to an English word several positions away, and the correspondence differs from pair to pair. *Self-attention* is the mechanism that lets a model relate each word to every other word in the sequence, regardless of position, rather than only to its immediate neighbors.

Concretely, a length- T input sequence $x^{(1)}, x^{(2)}, \dots, x^{(T)}$, each token embedded as a vector $x^{(i)} \in \mathbb{R}^d$, is turned into three parallel sequences of vectors via three learned projection matrices W_q, W_k, W_v :

$$q^{(i)} = W_q x^{(i)}, \quad k^{(i)} = W_k x^{(i)}, \quad v^{(i)} = W_v x^{(i)}, \quad q^{(i)}, k^{(i)}, v^{(i)} \in \mathbb{R}^{d_k},$$

for every position $i \in \{1, \dots, T\}$: a *query*, *key*, and *value* vector per token. How these three sequences combine into attention scores, and how those scores let one token gather information from every other token, is where we pick up next lecture — alongside the question this whole unit is building toward: what can an attacker learn about x from a trained $\mathcal{M}$ alone?

Looking ahead. We now have vocabulary for a trained model: an architecture, a loss, and the data that together produced it. Next lecture turns to attacks on that pipeline. The two questions from

Section 3.1 — was a given record used to train $\mathcal{M}$, and can properties of the training set be inferred from $\mathcal{M}$ alone — become *membership inference* and *property inference* attacks, mounted against exactly the kinds of models introduced here. Only once we have seen what a model can leak do we return to the question this lecture opened with: a *general* criterion for privacy, rather than a defense built against one attack at a time.

Key points

- **k -anonymity constrains the form of data.** A table can be k -anonymous and still let an attacker with side information (“everyone in this group has cancer”) learn a sensitive value with certainty.
- **Composition breaks k -anonymity.** Two separately k -anonymous releases of the *same* population can be cross-referenced to identify a sensitive value that neither release exposes on its own.
- **A machine learning model is a function fit to data** by an architecture (which function class), a loss (how the data picks one function from that class), and the training data itself. All three shape what the finished model can leak.
- **Discriminative models predict a label from a sample; generative models produce a sample from noise**, optionally conditioned on an input such as a text prompt.
- **The transformer computes a query, key, and value vector per token** via learned projections; how these combine into attention is next lecture’s starting point, alongside the attacks that motivate needing to understand it.

Further reading

- Sweeney, *k-Anonymity: A Model for Protecting Privacy* [5]: the original definition.
- Narayanan and Shmatikov, *Robust De-anonymization of Large Sparse Datasets* [4]: the Netflix Prize linkage attack that motivates k -anonymity’s goal in the first place.
- Ganta, Kasiviswanathan, and Smith, *Composition Attacks and Auxiliary Information in Data Privacy* [1]: the cross-referencing attack of Section 2.2, in full generality.
- Machanavajjhala, Kifer, Gehrke, and Venkatasubramanian, *ℓ -Diversity* [3], and Li, Li, and Venkatasubramanian, *t -Closeness* [2]: the two standard patches to the side-information failure.
- Vaswani et al., *Attention Is All You Need* [6]: the transformer architecture.
- Sebastian Raschka, *Self-Attention from Scratch*: <https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html>.

Acknowledgement

The k -anonymity material is adapted from course material jointly developed by Jonathan Ullman and Adam Smith. The translation example and self-attention illustration are adapted from Sebastian Raschka’s tutorial on self-attention, <https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html>. Corrections and suggestions are welcome.

References

- [1] Srivatsava Ranjit Ganta, Shiva Prasad Kasiviswanathan, and Adam Smith. Composition attacks and auxiliary information in data privacy. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '08, pages 265–273. ACM, 2008.
- [2] Ninghui Li, Tiancheng Li, and Suresh Venkatasubramanian. t -closeness: Privacy beyond k -anonymity and ℓ -diversity. In *IEEE 23rd International Conference on Data Engineering*, pages 106–115, 2007.
- [3] Ashwin Machanavajjhala, Daniel Kifer, Johannes Gehrke, and Muthuramakrishnan Venkitasubramaniam. ℓ -diversity: Privacy beyond k -anonymity. *ACM Transactions on Knowledge Discovery from Data*, 1(1), 2007.
- [4] Arvind Narayanan and Vitaly Shmatikov. Robust de-anonymization of large sparse datasets. In *IEEE Symposium on Security and Privacy*, pages 111–125, 2008.
- [5] Latanya Sweeney. k -anonymity: A model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5):557–570, 2002.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, NeurIPS '17, pages 5998–6008, 2017.